

Experiment 8

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	Semester	III
Course	Skill Lab: Python Programming	Course Code	VDCOR2VS201
Class	SY VLSI	Course In-Charge	Mr. Nirmol Munvar
Exp Name	Polymorphism and Inheritance		
CO	Apply object-oriented programming concepts such as classes, inheritance, and polymorphism to develop modular Python applications.		

Inheritance

Inheritance is an object-oriented programming feature in which a new class acquires the properties and methods of an existing class.

The existing class is called the **parent class** or **base class**, while the new class is called the **child class** or **derived class**.

Inheritance promotes **code reusability** because common functionality can be defined in the parent class and reused by child classes.

Basic syntax:

class Parent:

```
# parent class members
pass
```

class Child(Parent):

```
# child class members
pass
```

Example: Student and Engineering Student

A general Student class can contain common student information, while an EngineeringStudent class can inherit these properties and add engineering-specific information.

class Student:

```
def __init__(self, name, roll_no):
    self.name = name
    self.roll_no = roll_no
```

```
def display_student(self):
    print("Name:", self.name)
    print("Roll No:", self.roll_no)
```

class EngineeringStudent(Student):

```
def __init__(self, name, roll_no, branch):
    super().__init__(name, roll_no)
    self.branch = branch
```

```
def display_branch(self):  
    print("Branch:", self.branch)
```

```
student = EngineeringStudent("Rahul", 101, "VLSI")
```

```
student.display_student()  
student.display_branch()
```

Output:

Name: Rahul

Roll No: 101

Branch: VLSI

Here, EngineeringStudent inherits the display_student() method and the name and roll_no variables from Student.

Constructor with Inheritance

A child class can have its own constructor. The super() function can be used to call the constructor of the parent class.

```
class Student:
```

```
    def __init__(self, name, roll_no):  
        self.name = name  
        self.roll_no = roll_no
```

```
class EngineeringStudent(Student):
```

```
    def __init__(self, name, roll_no, branch):  
        super().__init__(name, roll_no)  
        self.branch = branch
```

```
student = EngineeringStudent("Priya", 102, "Electronics")
```

```
print(student.name)  
print(student.roll_no)  
print(student.branch)
```

Here, super().__init__(name, roll_no) calls the constructor of the Student class.

Types of Inheritance

Single inheritance occurs when one child class inherits from one parent class.

```
class Student:
```

```
    pass
```

```
class EngineeringStudent(Student):
```

```
    pass
```

Multilevel inheritance occurs when inheritance takes place across multiple levels.

```
class Student:
```

```
    pass
```

```
class EngineeringStudent(Student):
```

```
    pass
```

```
class VLSIStudent(EngineeringStudent):
```

```
    pass
```

Here, VLSIStudent inherits from EngineeringStudent, which in turn inherits from Student.

Multiple inheritance occurs when one child class inherits from multiple parent classes.

```
class DigitalDesign:
```

```
    def design(self):
```

```
        print("Digital Design")
```

```
class Verification:
```

```
    def verify(self):
```

```
        print("Design Verification")
```

```
class VLSIEngineer(DigitalDesign, Verification):
```

```
    pass
```

```
engineer = VLSIEngineer()
```

```
engineer.design()
```

```
engineer.verify()
```

The VLSIEngineer class inherits functionality from both DigitalDesign and Verification.

Polymorphism

Polymorphism means "many forms." In Python, polymorphism allows the same method or operation to behave differently depending on the object on which it is used.

In VLSI, a practical example is having different types of design blocks that all perform a simulate() operation, but each block performs the simulation differently.

Example: VLSI Design Blocks

Consider different VLSI blocks such as an ALU, memory, and multiplexer. Each block can have a simulate() method, but the implementation of the method differs for each block.

```
class ALU:
```

```
    def simulate(self):
```

```
        print("Simulating ALU operations")
```

```
class Memory:
```

```
    def simulate(self):
```

```
        print("Simulating memory read/write operations")
```

```
class Multiplexer:
```

```
    def simulate(self):
```

```
        print("Simulating multiplexer selection")
```

```
def run_simulation(design):
```

```
    design.simulate()
```

```
alu = ALU()
```

```
memory = Memory()
mux = Multiplexer()
```

```
run_simulation(alu)
run_simulation(memory)
run_simulation(mux)
```

Output:

Simulating ALU operations

Simulating memory read/write operations

Simulating multiplexer selection

The `run_simulation()` function uses the same statement:

```
design.simulate()
```

However, the behavior changes depending on whether the object is an ALU, Memory, or Multiplexer.

This is an example of **polymorphism** because the same method call, `simulate()`, produces different behavior for different objects.

Method Overriding

Polymorphism is also commonly implemented through **method overriding**, where a child class provides its own implementation of a method inherited from a parent class.

For example, different VLSI design blocks can inherit from a common `DesignBlock` class.

```
class DesignBlock:
```

```
    def simulate(self):
        print("Generic design simulation")
```

```
class ALU(DesignBlock):
```

```
    def simulate(self):
        print("Simulating ALU")
```

```
class Memory(DesignBlock):
```

```
    def simulate(self):
        print("Simulating Memory")
```

```
class RegisterFile(DesignBlock):
```

```
    def simulate(self):
        print("Simulating Register File")
```

```
blocks = [ALU(), Memory(), RegisterFile()]
```

```
for block in blocks:
```

```
    block.simulate()
```

Output:

Simulating ALU

Simulating Memory

Simulating Register File

All three classes inherit from `DesignBlock`, but each class **overrides** the `simulate()` method with its own implementation.

Thus, the same method call:

```
block.simulate()
```

produces different behavior depending on the object.

Inheritance and Polymorphism Together

Inheritance provides the mechanism for **reusing common functionality**, while polymorphism allows different child classes to provide **different implementations of common operations**.

A VLSI verification example can illustrate both concepts:

```
class VerificationComponent:
```

```
    def verify(self):
        print("Generic verification")
```

```
class ALUVerification(VerificationComponent):
```

```
    def verify(self):
        print("Verifying ALU operations")
```

```
class MemoryVerification(VerificationComponent):
```

```
    def verify(self):
        print("Verifying memory read/write")
```

```
class APBVerification(VerificationComponent):
```

```
    def verify(self):
        print("Verifying APB transactions")
```

```
components = [
    ALUVerification(),
    MemoryVerification(),
    APBVerification()
]
```

```
for component in components:
```

```
    component.verify()
```

Output:

Verifying ALU operations

Verifying memory read/write

Verifying APB transactions

Here, ALUVerification, MemoryVerification, and APBVerification **inherit** from VerificationComponent.

At the same time, they each **override** the verify() method, demonstrating polymorphism.

Basic Exercises

Exercise 1: Student Inheritance

Create a Student parent class with:

- name
- roll_no
- display_student() method

Create an EngineeringStudent child class with:

- branch
- semester
- display_branch() method

Use super() to initialize the parent class variables. Create two engineering student objects and display their complete details.

Exercise 2: VLSI Design Inheritance

Create a parent class VLSIBlock with:

- name
- technology
- display_info() method

Create two child classes:

- Processor
- Memory

Add one additional instance variable specific to each child class and display the complete information of each object.

Exercise 3: VLSI Polymorphism

Create a parent class DesignBlock with a method simulate().

Create the following child classes:

- ALU
- RegisterFile
- Cache

Override the simulate() method in each child class to display an appropriate simulation message.

Store objects of all three classes in a list and use a for loop to call simulate() for every object.

Exercise 4: VLSI Verification Polymorphism

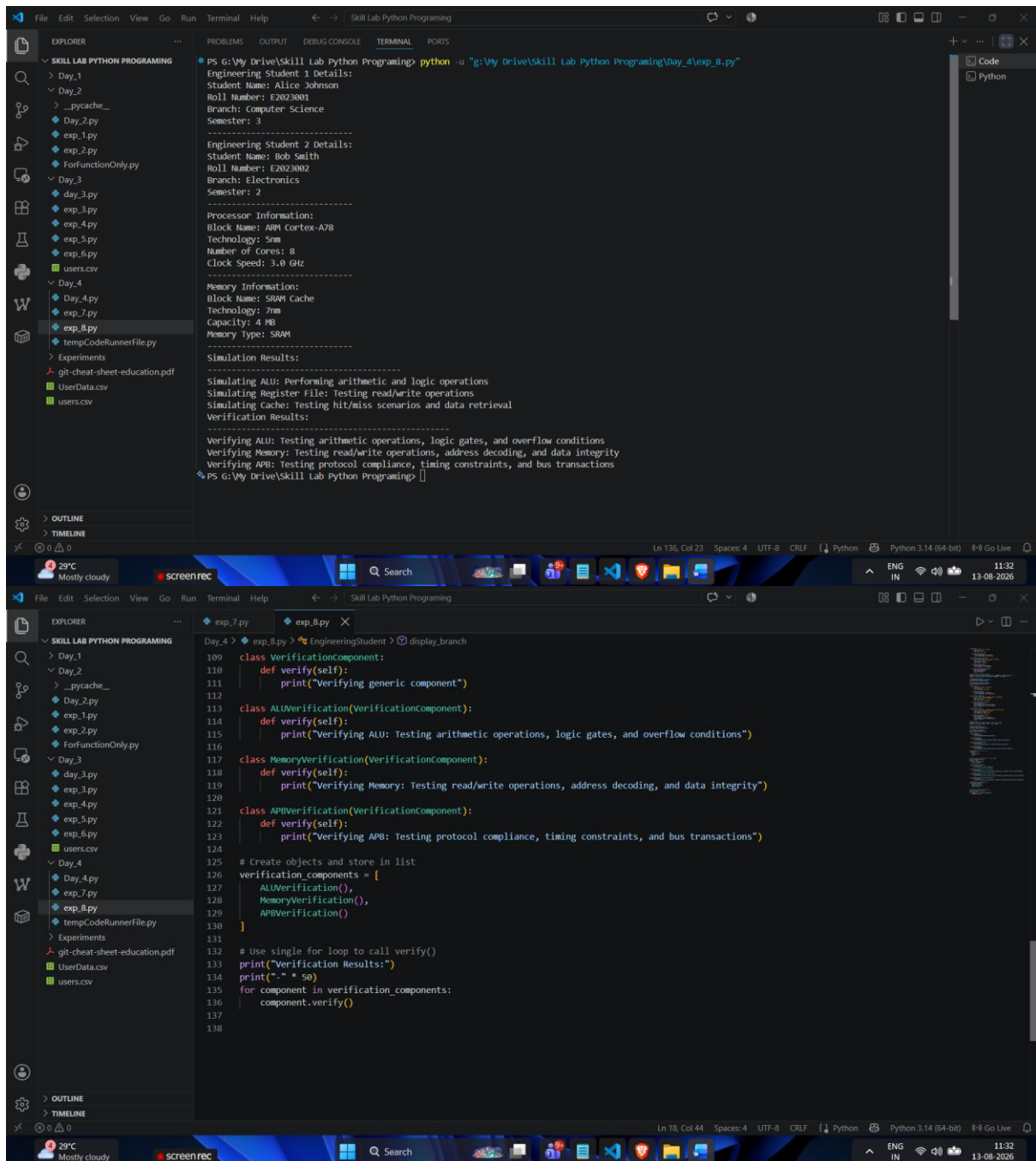
Create a parent class VerificationComponent with a method verify().

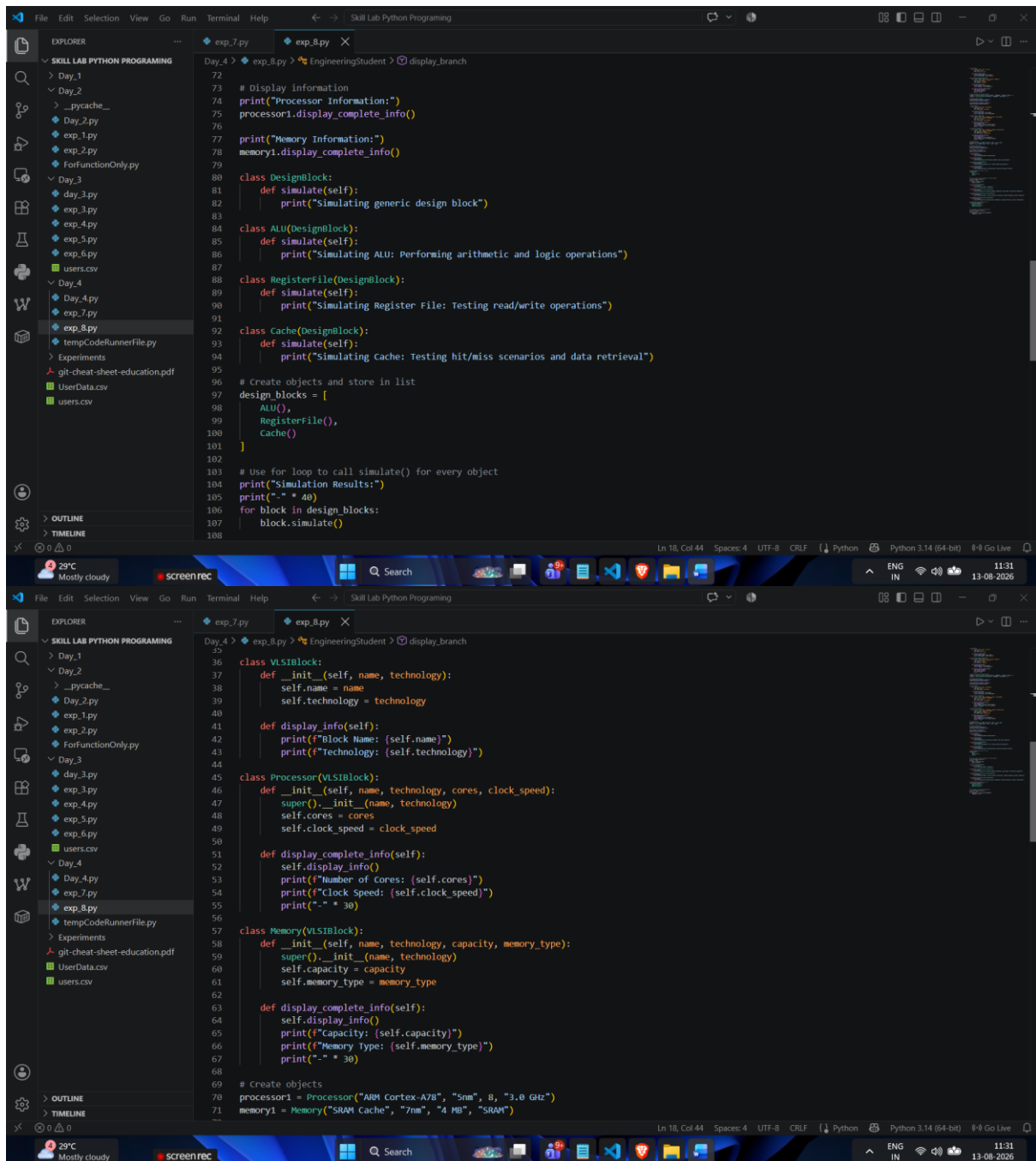
Create the following child classes:

- ALUVerification
- MemoryVerification
- APBVerification

Override verify() in each class to display what is being verified.

Create a list containing objects of all three classes and use a single for loop to call verify().






```
File Edit Selection View Go Run Terminal Help Skill Lab Python Programming
EXPLORER
SKILL LAB PYTHON PROGRAMING
  Day_1
  Day_2
  __pycache__
  Day_2.py
  exp.1.py
  exp.2.py
  ForFunctionOnly.py
  Day_3
    day.3.py
    exp.3.py
    exp.4.py
    exp.5.py
    exp.6.py
    users.csv
  Day_4
    Day_4.py
    exp.7.py
    exp.8.py
    tempCodeRunnerFile.py
  Experiments
  git-cheat-sheet-education.pdf
  UserData.csv
  users.csv
OUTLINE
TIMELINE
Ln 18, Col 44 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live
1 class Student:
2     def __init__(self, name, roll_no):
3         self.name = name
4         self.roll_no = roll_no
5
6     def display_student(self):
7         print(f"Student Name: {self.name}")
8         print(f"Roll Number: {self.roll_no}")
9
10 class EngineeringStudent(Student):
11     def __init__(self, name, roll_no, branch, semester):
12         super().__init__(name, roll_no)
13         self.branch = branch
14         self.semester = semester
15
16     def display_branch(self):
17         print(f"Branch: {self.branch}")
18         print(f"Semester: {self.semester}")
19
20     def display_complete_details(self):
21         self.display_student()
22         self.display_branch()
23         print("." * 30)
24
25 # Create engineering student objects
26 student1 = EngineeringStudent("Alice Johnson", "E2023001", "Computer Science", 3)
27 student2 = EngineeringStudent("Bob Smith", "E2023002", "Electronics", 2)
28
29 # Display their details
30 print("Engineering Student 1 Details:")
31 student1.display_complete_details()
32
33 print("Engineering Student 2 Details:")
34 student2.display_complete_details()
35
36 class VLSIBlock:
37     def __init__(self, name, technology):
```